



FLACSO
ECUADOR

SESIÓN 1: INGESTA Y ARQUITECTURA

Ingesta de Datos y Arquitectura de Tablas

Fausto Jácome Pérez

FLACSO Ecuador

Vamos a trabajar con la Encuesta de Desnutrición Infantil (ENDI)

22.250

VIVIENDAS VISITADAS

93.242

PERSONAS REGISTRADAS

6

TABLAS RELACIONALES

Población objetivo: Niñas y niños menores de 5 años

Cobertura: Nacional, Regional, Provincial, Urbano–Rural

Comparabilidad histórica: ECV 2006/2014, ENSANUT 2012/2018, ENDI 2022–23

Una encuesta así **no cabe en una sola tabla**. Vamos a entender por qué.

Datos estructurados, semiestructurados y no estructurados

¿Dónde entra la ENDI?

Tipos de datos según su formato y reglas esquema

Estructurados

Filas y columnas. Esquema fijo. SQL nativo.

Ejemplos: encuesta, registros administrativos, contabilidad, bancarios.

Formatos: csv*, xlsx*, parquet, sql

La ENDI entra aquí

Semiestructurados

Tienen estructura, pero flexible. No hay esquema rígido.

Ejemplos: logs de servidor, respuestas de API ([Clima](#)), Geography Markup Language (GML) [ver mapa]

Formatos: csv*, xlsx*, JSON, XML, YAML

No estructurados

Sin esquema predefinido. Requieren procesamiento extra.

Ejemplos: texto libre, imágenes, audio, video.

Este tramo se enfoca en **datos estructurados**. La misma lógica de organización aplica a los otros tipos, pero el punto de partida más común en ciencias sociales es la tabla.

¿Por qué separar en tablas?

Normalización y formas normales

El problema de la tabla única

Imaginemos que guardamos todo en una sola tabla: vivienda, hogar, personas y niños.

¿Qué problemas aparecen?

- **Redundancia:** los datos de la vivienda se repiten para cada persona que vive ahí
- **Anomalías de actualización:** si hay que modificar un dato de vivienda, hay que actualizar N filas
- **Anomalías de eliminación:** si se hay que eliminar a una persona en una vivienda unipersonal , perdemos todos los datos de la vivienda
- **Tamaño:** la tabla crece innecesariamente

La solución es normalización: Organizar los datos separando la información en tablas según su unidad de análisis.

Las **formas normales** (1FN, 2FN, 3FN) son las reglas formales de ese proceso. No las derivamos aquí, pero el principio es simple: *cada tabla describe una sola cosa*. ([ver más detalles](#)).

Modelo lógico de la ENDI

Diagrama entidad-relación simplificado



Llaves primarias 🔑 - identifican de forma única cada fila

Llaves foráneas 🔗 - conectan tablas entre sí

Modelo lógico → qué entidades existen y cómo se relacionan

Modelo físico → cómo se implementa en PostgreSQL (sesión 3–4)

Las 6 tablas de la ENDI

Tabla	Unidad de análisis	Filas	Factor de expansión
f1_personas	Miembro del hogar	93 242	fexp
f1_hogar	Hogar	20 186	fexp
f2_mef	Mujer en edad fértil (10–49 años)	19 846	fexp
f2_lactancia	Niño/a menor de 3 años	12 141	fexp_lm
f2_salud_ninez	Niño/a menor de 5 años	22 188	fexp
f3_desarrollo_inf	Niño/a menor de 5 años	9 836	fexp_di

⚠ Cada tabla tiene **su propio factor de expansión**. Mezclarlos es uno de los errores más comunes al trabajar con encuestas. Lo profundizamos en sesión 4.

El flujo del dato

Desde el archivo en disco hasta el resultado para el análisis

Disco → RAM → CPU

Disco duro

El archivo existe aquí.

``'.csv`, `'.rds`, `'.dta``

No se puede calcular nada hasta que se cargue.

RAM (memoria)

La tabla vive aquí mientras trabajamos.

Polars / R / Pandas la cargan acá.

Si la tabla es más grande que la RAM → problema.

CPU

Aquí se ejecutan las funciones: filtros, agrupaciones, joins.

Polars lo paraleliza por columna.

Cuando hacemos `pl.read_csv("archivo.csv")` estamos **moviendo datos del disco a la RAM**. Lo que viene después ocurre en la RAM y el CPU.

¿Por qué Polars y no Pandas? ¿Por qué Postgres no explota?

Pandas vs Polars

Pandas carga toda la tabla entera en RAM, en formato de filas. Cada operación recorre fila por fila.

Polars usa almacenamiento **columnar** (Arrow) y **evaluación perezosa** (*lazy evaluation*):

- Construye un plan de consulta
- Solo lee las columnas necesarias
- Ejecuta en paralelo por columna
- Para datos que no caben en RAM, Polars tiene un modo streaming que procesa en lotes.

Resultado: Polars puede ser 5–20× más rápido en operaciones de agregación.

¿Por qué Postgres no se "cuelga" con millones de filas?

- Los datos viven en **disco** en páginas de 8KB, no en RAM
- Los **índices** evitan el escaneo completo: recorren un árbol hasta encontrar la página + slot exacto donde vive cada fila ($O(\log n)$ en lugar de $O(n)$)
- Sin índice, Postgres hace un *sequential scan* (lee todas las páginas)
- En nuestro caso, las llaves primarias (`id_per` , `id_hogar`) tienen índice automático

Postgres puede analizar una tabla de 50M de filas en un servidor con 8GB de RAM porque nunca necesita cargarla completa.

Formatos de entrada

Los datos no siempre llegan como uno quisiera

Formatos comunes y sus particularidades

Texto plano (¿simple?)

.csv - separado por comas (*¡ojo con el separador!*)

- Puede ser `;` en configuraciones en español
- Puede ser `\t` (TSV)
- **Encoding:** `UTF-8` o `latin-1` / `ISO-8859-1` - crítico para tildes y ñ

¿Qué es el encoding? Es la tabla que mapea caracteres a números binarios. UTF-8 puede representar cualquier carácter del mundo. Latin-1 solo cubre el alfabeto occidental (default de Windows en español).

```
# CSV con separador punto y coma
import polars as pl
pl.read_csv("archivo.csv", separator=";",
            encoding="latin-1")
```

Formatos estadísticos

.rds - formato nativo de R, preserva tipos

.dta - formato Stata

.sav - formato SPSS

.xlsx - Excel, ojo con hojas múltiples

```
# Leer .rds directamente desde Python
import pyreadr
import polars as pl

resultado = pyreadr.read_r("tabla.rds")
df = pl.from_pandas(resultado[None])
# None porque el rds tiene un solo objeto
```

El INEC ofrece la ENDI en `.rds` y `.dta`. En clase descargaremos en `.rds` ya que preserva los tipos de variable automáticamente.

Piedra Rosetta #1

Cargar datos: la misma operación en tres herramientas

Piedra Rosetta - Carga desde CSV

Python - Polars

```
import polars as pl

df = pl.read_csv(
    "BDD_ENDI_R2_f1_personas.csv",
    separator=";",
    encoding="utf-8",
    has_header=True
)
```

R

```
library(readr)

df <- read_delim(
    "BDD_ENDI_R2_f1_personas.csv",
    delim = ";",
    locale = locale(encoding = "UTF-8"),
    col_names = TRUE
)
```

SQL - PostgreSQL

```
# Debe existir el DDL previamente (próximas clases)

COPY personas
FROM '/ruta/BDD_f1_personas.csv'
DELIMITER ';'
CSV HEADER
ENCODING 'UTF8';
```

Piedra Rosetta: las tres operaciones hacen exactamente lo mismo. El concepto - *leer un archivo e inferir su estructura* independientemente de la herramienta.

read_csv VS scan_csv (lazy)

read_csv - ejecución inmediata

```
df = pl.read_csv(  
    "BDD_ENDI_R2_f1_personas.csv",  
    separator=";",  
    encoding="utf-8",  
    has_header=True  
)
```

- Lee el archivo completo al llamar la función
- Las 124 columnas y 93.242 filas entran a RAM de una vez
- El resultado es un `DataFrame` listo para usar

scan_csv - ejecución perezosa

```
df = (  
    pl.scan_csv(  
        "BDD_ENDI_R2_f1_personas.csv",  
        separator=";",  
        encoding="utf-8",  
        has_header=True  
    )  
    .filter(pl.col("area") == 1)  
    .select(["id_per", "fexp", "area"])  
    .collect()  
)
```

- `scan_csv` no lee nada todavía - construye un plan
- `.filter()` y `.select()` agregan instrucciones al plan
- `.collect()` ejecuta todo: lee solo las columnas y filas necesarias
- El resultado es el mismo `DataFrame`, pero Polars nunca cargó las 124 columnas completas

Piedra Rosetta - Carga desde RDS

Python - Polars

```
import pyreadr
import polars as pl

df = pl.from_pandas(
    pyreadr.read_r(
        "BDD_ENDI_R2_f1_personas.rds"
    )[None]
)
```

R

```
df <- readRDS(
  "BDD_ENDI_R2_f1_personas.rds"
)
```

En R la carga desde `.rds` es nativa y preserva los tipos de variable. En Python requiere convertir a pandas como paso intermedio.

Tipos de datos

Más allá de "numérico" y "categórico"

Del concepto al byte

La distinción cuantitativo/cualitativo es analítica. Los tipos de datos en memoria son más específicos:

Tipo	¿Qué guarda?	Bytes típicos	Ejemplo en ENDI
Int8 / Int16 / Int32 / Int64	Enteros	1–8 bytes	edad, número de hijos
Float32 / Float64	Decimales	4–8 bytes	talla, peso, fexp
Utf8 / String	Texto	Variable	nombre de provincia
Boolean	Verdadero / Falso	1 bit	¿tiene agua potable?
Date / Datetime	Fechas	4–8 bytes	fecha de entrevista
Categorical	Texto con categorías fijas	Comprimido	área (urbano/rural)

Precisión importa: un Float64 ocupa el doble que un Float32 . En una tabla de 93.000 filas y 124 columnas, usar el tipo correcto puede reducir el uso de RAM a la mitad.

Bits, memoria y rangos

Con **N bits** se pueden representar **2^N combinaciones**. Un bit reservado para el signo parte ese espacio en dos: mitad para negativos, mitad para positivos.

Enteros

Tipo	Bits	Rango
Int8	8	-128 a 127
UInt8	8	0 a 255
Int16	16	-32.768 a 32.767
Int32	32	-2.147M a 2.147M
Int64	64	-9.2×10^18 a 9.2×10^18
Boolean	1	True / False

Decimales (punto flotante - IEEE 754)

Tipo	Bits	Signo	Exponente	Mantisa	Precisión
Float32	32	1	8	23	~7 dígitos
Float64	64	1	11	52	~15 dígitos

1

s

8 bits

e

23 bits

m

$$a = (-1)^s \cdot 1.m_2 \cdot 2^{e_2-127}$$

fexp en la ENDI es Float64 - necesita los 15 dígitos de precisión para no distorsionar las estimaciones ponderadas.

Los tipos en la ENDI - Polars

```
import polars as pl

df = pl.read_csv("BDD_ENDI_R2_f1_personas.csv", separator=";")

# Ver nombres de columnas y sus tipos
print(df.schema)

# Alternativa: ver tipos como tabla
print(df.dtypes)          # lista de tipos en orden
print(df.columns)         # lista de nombres en orden

# Combinarlos como tabla legible
import pandas as pd       # solo para mostrar bonito
pd.DataFrame({"columna": df.columns, "tipo": df.dtypes})
```

Polars infiere los tipos automáticamente al leer el CSV. Pero siempre vale la pena **verificar**: una columna de códigos numéricos (ej. código de provincia) puede ser leída como `Int64` cuando debería tratarse como `String` o `Categorical`.

Exploración inicial

Variables, tipos, resumen y missings

Piedra Rosetta #2 - Exploración inicial

Python · Polars

```
# Forma y primeras filas
df.shape          # (filas, cols)
df.head(5)
df.tail(3)

# Esquema
df.schema

# Resumen estadístico
df.describe()
```

R · base / dplyr

```
# Forma
dim(df)           # filas, cols
head(df, 5)

# Tipos
str(df)
glimpse(df)       # dplyr

# Resumen
summary(df)
```

SQL · PostgreSQL

```
-- Contar filas
SELECT COUNT(*) FROM personas;

-- Ver primeras filas
SELECT * FROM personas
LIMIT 5;

-- Columnas y tipos
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'personas';

\d personas -- en psql
```

Resumen numérico y categorías

Variables numéricas

```
# Resumen completo: media, min, max,  
# std, percentiles  
df.describe()  
  
# Solo columnas numéricas  
df.select(pl.col(pl.NUMERIC_DTYPES)).describe()  
  
# Distribución de una variable  
df["p3_2"].value_counts().sort("count",descending=True)
```

`describe()` devuelve: `count`, `null_count`, `mean`, `std`,
`min`, `25%`, `50%`, `75%`, `max`

Variables categóricas

```
# Categorías únicas de una variable  
df["area"].unique()  
  
# Conteo de cada categoría  
df["area"].value_counts()  
  
# Para varias columnas a la vez  
cat_cols = ["area", "region", "parr_pri"]  
for col in cat_cols:  
    print(f"\n— {col} —")  
    print(df[col].value_counts())
```

Datos perdidos - primer conteo

¿Cuántos nulos hay?

```
# Conteo de nulos por columna
df.null_count()

# Proporción de nulos
(df.null_count() / len(df) * 100).transpose(
    include_header=True,
    column_names=["pct_nulo"]
).sort("pct_nulo", descending=True)
```

Tipos de ausencia en la ENDI

Código	Significa
NA / .	Ausente por flujo del formulario
88	No sabe / No responde
99 / 9999	Omisión del informante
999	Limitación logística

En la ENDI, los missings **no son aleatorios**. Un 'NA' en la sección de lactancia significa que ese niño tenía más de 3 años. Interpretarlos mal cambia los resultados.

¿Qué aprendimos hoy?

Conceptos

- Datos estructurados, semiestructurados, no estructurados
- Normalización: por qué una tabla por entidad
- Diagrama ER, modelo lógico vs físico
- El flujo disco → RAM → CPU
- Tipos de datos: `int`, `float`, `string`, `boolean`, precisión

En Polars

- `pl.read_csv()` con separador y encoding
- `.shape`, `.schema`, `.dtypes`, `.columns`
- `.describe()` para variables numéricas
- `.value_counts()` para categóricas
- `.null_count()` para missings

Próxima sesión: tenemos los datos cargados - ahora los transformamos. Data wrangling: filtros, recodificación y reshaping (long vs wide).

Ejercicio práctico

Tarea de la hora práctica: exploración inicial de la tabla `f1_personas` de la ENDI

Instrucciones:

1. Carguen la tabla `BDD_ENDI_R2_f1_personas.rds` y transformen a Polars
2. Impriman la forma de la tabla (filas y columnas)
3. Muestren el schema completo (nombre + tipo de cada columna)
4. Ejecuten `.describe()` y identifiquen 3 variables numéricas de interés
5. Hagan `.value_counts()` sobre las variables `area` y `region`
6. Calculen el porcentaje de nulos por columna - ¿cuáles tienen más missings?

Pregunten: ¿qué tipo de dato tiene `fexp` ? ¿Tiene sentido ese tipo? ¿Y el código de provincia `prov` ?

Sin tarea para el jueves. Sí habrá evaluación diaria.